

SPIFFE + SPIRE mTLS Demo

File-by-file explanation of the current `spiffe-spire-mtls-demo` repository

PROJECT FLOW

01

Docker creates the environment

02

SPIRE Server establishes the trust domain

03

SPIRE Agent becomes trusted

04

Agent attests workloads

05

Registration entries map selectors to SPIFFE IDs

06

Workloads retrieve X.509-SVIDs

07

Node.js uses those credentials for mTLS

08

The API authorizes the authenticated caller by SPIFFE ID

01

`docker-compose.yml`

Wires the Lab Together

This is the orchestration file. It defines the SPIRE Server, SPIRE Agent, authorized Client, Rogue Client, and API containers and connects the pieces needed for workload identity.

It mounts the Server and Agent configuration, gives the Agent access to Docker metadata through `/var/run/docker.sock`, shares the Agent Workload API socket with all workloads, and assigns the Docker labels used by the Docker WorkloadAttestor.

Client label	<code>spiffe.workload: "client"</code>
Rogue Client label	<code>spiffe.workload: "rogue-client"</code>
API label	<code>spiffe.workload: "api"</code>

WHY THIS MATTERS

The authorized Client and Rogue Client reuse the same `./client` application code. Their different identities come from observed Docker labels plus SPIRE registration policy, not from code that claims a SPIFFE ID.

MENTAL SHORTCUT

`docker-compose.yml` creates and connects the environment; it does not itself grant SPIFFE identities.

Configures the Identity Authority

This configures the SPIRE Server, the central authority for the demo. It establishes `demo.local` as the trust domain, listens for Agents on TCP 8081, configures the SQLite datastore, enables join-token node attestation, and stores Server signing keys on disk inside the container.

```
trust_domain = "demo.local"
```

Workload SPIFFE IDs are assigned through registration entries, not `server.conf`.

The join-token NodeAttestor is why the demo generates a one-time token when bootstrapping the Agent. That establishes trust in the Agent before it can participate in workload identity issuance.

MENTAL SHORTCUT

`server.conf` answers: Who controls the trust domain? How does an Agent join? Where is Server state stored? How are signing keys managed?

Configures the Workload Identity Broker

This configures the SPIRE Agent. It tells the Agent which trust domain it belongs to, where the Server is, where to expose the local Workload API, and which plugins to use for node and workload attestation.

```
server_address    = "spire-server"  
server_port      = "8081"  
socket_path      = "/run/spire/sockets/api.sock"  
insecure_bootstrap = true
```

The Agent uses the Unix and Docker workload attestors. The Docker attestor can inspect container metadata and derive selectors such as:

```
docker:label:spiffe.workload:client  
docker:label:spiffe.workload:rogue-client  
docker:label:spiffe.workload:api
```

The Agent uses an in-memory KeyManager. After the Agent is stopped, its key material is lost; a later bootstrap requires a fresh join token and can produce a different Agent SPIFFE ID. Workload entries must therefore be parented to the currently attested Agent.

MENTAL SHORTCUT

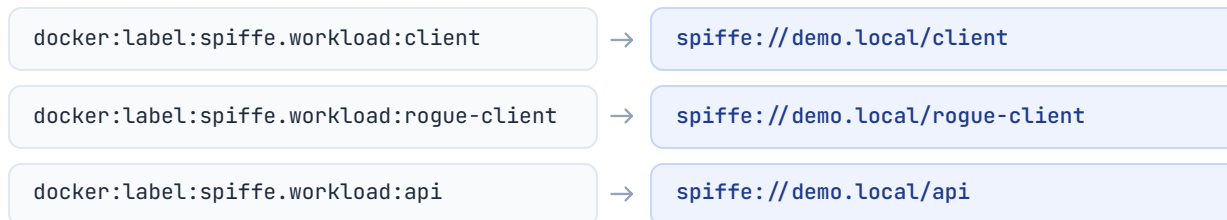
`agent.conf` answers: How does the Agent join SPIRE? How does it identify workloads? Where do workloads request identity?

04

Registration Entries

The Policy Mapping

Registration entries are created with SPIRE commands rather than stored as repository files, but they are essential to the runtime behavior. They map attested selectors to SPIFFE IDs under the currently attested Agent.



The workload never sends a request such as “give me `spiffe://demo.local/client`”. The Agent identifies the caller, derives selectors from observed properties, and registration policy determines the identity that can be returned.

CORE IDEA

Workload identity is derived from attested evidence and policy, not asserted by the application.

05

api/Dockerfile

Builds the API Workload

The API Dockerfile builds the TypeScript application and includes the SPIRE Agent CLI used by the entrypoint. The image contains everything needed to fetch an X.509-SVID before the Node.js process starts.



The Dockerfile does not assign `spiffe://demo.local/api`. That identity is determined by workload attestation and registration policy.

06

api/entrypoint.sh

Retrieves the API Identity

Before Node starts, the entrypoint calls the Agent Workload API and writes the API credential material into `/tmp`.

```
/tmp/svid.0.pem
/tmp/svid.0.key
/tmp/bundle.0.pem
```

These files are the X.509-SVID certificate, corresponding private key, and trust bundle. Because the script uses `set -e`, identity retrieval failure stops startup immediately.

MENTAL SHORTCUT

Get identity first; if SPIRE will not issue the API identity, the application does not start.

07

api/src/index.ts

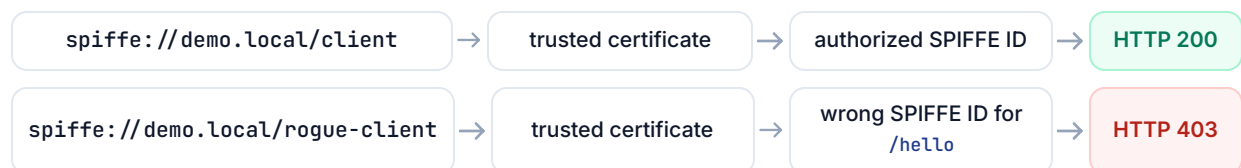
Authenticates and Authorizes Callers

This is the main v2 change. The Node.js/Express API uses its SPIRE-issued certificate and key for HTTPS, requires callers to present a client certificate, validates that certificate against the SPIFFE trust bundle, and then performs an application-level authorization check on the peer SPIFFE ID.

```
requestCert: true
rejectUnauthorized: true

AUTHORIZED_CLIENT_SPIFFE_ID =
  "spiffe://demo.local/client"
```

The API reads the peer certificate Subject Alternative Name, extracts the URI identity, and compares it with the one authorized Client identity. A valid but different SPIFFE identity reaches the HTTP layer but receives **403 Forbidden**.



Authentication is not authorization. A certificate can be valid and trusted while the authenticated workload is still not permitted to call the protected endpoint.

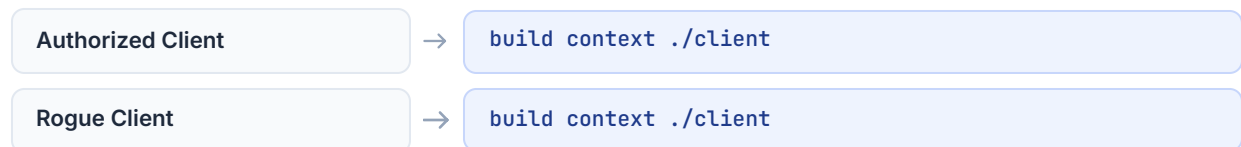
08 `api/package.json`, `package-lock.json`, `tsconfig.json`

`package.json` defines Node dependencies and build/start scripts. `package-lock.json` pins the exact dependency graph. `tsconfig.json` controls TypeScript compilation. These files support the API application but do not decide SPIFFE identity or authorization policy.

09 `client/Dockerfile`

Builds Both Client Workloads

The authorized Client and Rogue Client use the same Client Dockerfile, build context, and TypeScript application code. It also includes the SPIRE credential-fetching capability used by the entrypoint.



The image does not hardcode either `spiffe://demo.local/client` or `spiffe://demo.local/rogue-client`. The same program receives different identities because the running containers have different attested labels.

10 `client/entrypoint.sh`

Retrieves the Running Workload Identity

The Client entrypoint contacts the shared Workload API socket and asks for an X.509-SVID. It does not request a SPIFFE ID by name.



KEY CONCEPT

The identical application can become the authorized Client or the Rogue Client because identity follows from attested runtime properties and policy.

11 `client/src/index.ts`

Initiates the mTLS Request

The client application loads its X.509-SVID, private key, and trust bundle and initiates an HTTPS request to `api:3000/hello`. The same code runs for both Client containers.



The application now prints both the HTTP status and response body. This makes the authorization distinction obvious during the live demo: 200 for the authorized Client and 403 for the Rogue Client.

```
Authorized Client: API status: 200
Rogue Client:      API status: 403
```

The current client validates that the API certificate chains to the configured `demo.local` trust bundle, but it disables conventional DNS hostname checking and does not perform an exact client-side SPIFFE-ID check requiring `spiffe://demo.local/api`.

CURRENT SCOPE

Exact SPIFFE ID authorization is demonstrated on the API side. Client-side exact peer SPIFFE-ID authorization is not yet implemented.

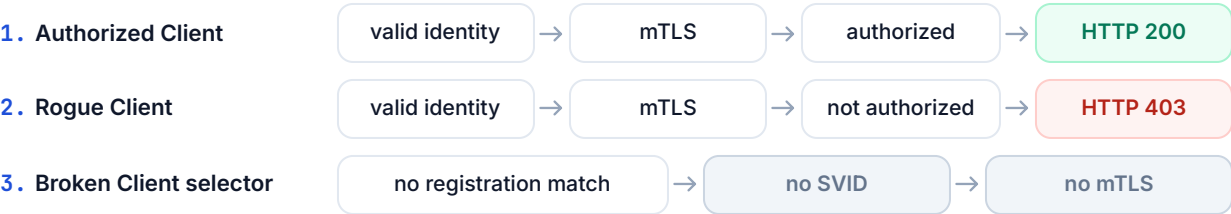
12 **client/package.json, package-lock.json, tsconfig.json**

These files define dependencies, lock exact versions, and configure TypeScript compilation for the shared Client application. They support both Client containers but do not determine which SPIFFE identity either container receives.

13 **README.md**

Documents the Complete v2 Demo

The README is the operational walkthrough and conceptual source of truth for the repository. It now documents all three workloads, all three registration entries, the clean bootstrap sequence, and the three runtime outcomes.



It also documents the current lab limitations: client-side exact API SPIFFE-ID authorization is not implemented, Agent key storage is in memory, `insecure_bootstrap = true` is used for the local lab, and the declared `spire-server-data` volume is not currently mounted into the Server container.

MENTAL SHORTCUT

The README explains how to reproduce the system from a clean environment and how to interpret each success or failure mode.

14 `.gitignore`

Keeps Repository Noise Out of Git

This prevents generated or local artifacts from being committed. It has no direct role in SPIFFE/SPIRE identity, authentication, or authorization.

15 `docs/SPIFFE_SPIRE_Project_File_Explanations.pdf`

This document is a companion explanation of the repository. It should be regenerated whenever the architecture or runtime behavior changes so it does not contradict the code or README.

This regenerated version reflects the Rogue Client and SPIFFE ID authorization enhancement.

Complete File-to-Flow Mental Model

- 1 `docker-compose.yml`
 - Creates Server, Agent, Client, Rogue Client, API
- 2 `server.conf`
 - Server controls `demo.local` and establishes Agent trust
- 3 `agent.conf`
 - Agent exposes Workload API and attests workloads
- 4 Docker labels + `WorkloadAttestors`
 - Agent derives selectors
- 5 Registration entries
 - `client` → `spiffe://demo.local/client`
 - `rogue-client` → `spiffe://demo.local/rogue-client`
 - `api` → `spiffe://demo.local/api`
- 6 `entrypoint.sh`
 - Each workload retrieves SVID + key + trust bundle
- 7 `client/src/index.ts`
`api/src/index.ts`
 - Node.js uses credentials for mTLS
 - API authorizes the authenticated caller by SPIFFE ID

RESPONSIBILITIES TO MEMORIZE

Compose creates the environment. Server config defines the identity authority. Agent config defines how the Agent joins and attests workloads. Registration entries map observed properties to identities. Entrypoints retrieve credentials. Application code uses those credentials for mTLS, and the API adds the authorization decision.

Three Outcomes to Be Able to Explain

Scenario	Identity issuance	mTLS authentication	Authorization / result
Authorized Client	Succeeds as <code>spiffe://demo.local/client</code>	Succeeds	Allowed · HTTP 200
Rogue Client	Succeeds as <code>spiffe://demo.local/rogue-client</code>	Succeeds	Denied · HTTP 403
Broken Client selector	Fails: no matching registration entry	Never begins	Application never starts

1. Authorized Client

- correct selector
- registration match
- `spiffe://demo.local/client`
- valid X.509-SVID
- mTLS succeeds
- API authorizes identity
- HTTP 200

2. Rogue Client

- rogue-client selector
- registration match
- `spiffe://demo.local/rogue-client`
- valid X.509-SVID
- mTLS succeeds
- API rejects identity
- HTTP 403

3. Broken Selector

- broken-client selector
- no registration match
- no SPIFFE ID
- no X.509-SVID
- entrypoint exits
- Node.js never starts
- mTLS never begins

THE PRESENTATION-LEVEL DISTINCTION

The Rogue Client proves *authentication can succeed while authorization fails*. The broken-selector test proves identity issuance can fail before authentication is even possible.

Final Mental Model



The workload does not choose its identity. SPIRE derives identity entitlement from observed workload properties and registration policy. The application can then use that cryptographically authenticated identity to decide whether a requested operation is authorized.